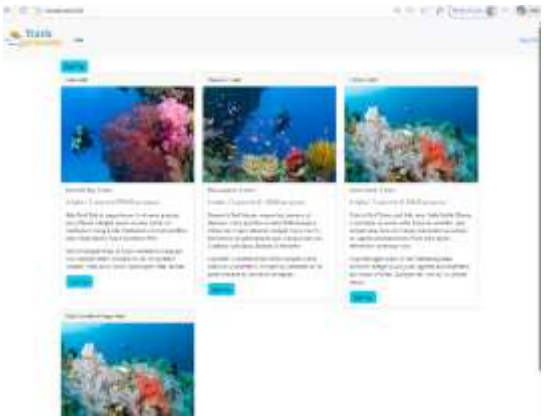
Capstone Project Narrative

The artifact I selected for this project is my Travlr Getaways web application, which I created in a previous course. This is a full-stack application, meaning it has a frontend (what users see), a backend (the server that processes requests), and a database (where data is stored). The purpose of the application is to allow users to view travel trips and allow administrators to add, edit, and manage those trips.

I chose this project for my ePortfolio because it shows a complete system working together. It also gave me the chance to go back and improve my original code. Instead of just building something new, I was able to practice improving existing code, which is something developers do a lot in real jobs.



When I reviewed my code, I noticed that it worked, but it could be cleaner, safer, and easier to understand. So I focused on improving the backend part of the project, especially on how users log in and how the system handles requests. Below are some of the samples of the changes I made for Part 1:

Fixing User Registration:

Before: After:

```
const user = new User({
  name: req.body.name,
  email: req.body.email,
  password: ''
});
user.setPassword(req.body.password)
const q = await user.save();

if(!q)
{
  return res.status(400).json(err);
}
```

```
const existingUser = await User.findOne({ email }).exec();
if (existingUser) {
  return res.status(409).json({
    message: "User already exists"
  });
}

const user = new User({ name, email });
user.setPassword(password);
await user.save();

const token = user.generateJWT();
return res.status(200).json({ token });
```

What was wrong:

- It didn't check if the user already existed
- It didn't handle errors clearly
- The code was harder to read

Improvement:

- It prevents duplicate users
- It gives clearer error messages
- It is easier to read and understand

Fixing Security JWT authentication

Before After

```
jwt.verify(token, process.env.JWT_SECRET, (err, verified) => {
    if(err) {
        return res.sendStatus(401).json('Token Validation Error!');
    }
    req.auth = verified;
});
next();
```

```
jwt.verify(token, process.env.JWT_SECRET, (err, verified) => {
  if (err) {
    return res.status(401).json({
      message: "Token validation error"
    });
  }

  req.auth = verified;
  next();
});
```

What was wrong:

- Incorrect response handling

- The system allowed access even if the token was invalid

What is better:

- Only valid users can access protected parts

Completing the API

Before

```
router
  .route('/trips/:tripCode')
  .get(tripsController.tripsFindByCode)
  .put(authenticateJWT, tripsController.tripsUpdateTrip);
```

After

```
router
  .route('/trips/:tripCode')
  .get(tripsController.tripsFindByCode)
  .put(authenticateJWT, tripsController.tripsUpdateTrip)
  .delete(authenticateJWT, tripsController.tripsDeleteTrip);
```

What was wrong:

- There was no way to delete a trip

What is better:

- Now the system supports full CRUD (Create, Read, Update, Delete)

- Makes the app completer and more realistic

List of files updated:

- travlr/app.js

- travlr/app_api/models/db.js

- travlr/app_api/config/passport.js

- travlr/app_api/controllers/authentication.js

- travlr/app_api/controllers/trips.js

- travlr/app_api/routes/index.js
- travlr/app_api/models/user.js

When looking at the Data structure, I noticed that the application was simply returning all trips at once without any filtering, sorting, or organization. While this worked for small datasets, it would not scale well as more data is added. To improve this, I added search, sorting, and pagination features to better manage how trip data is processed and displayed. Below are some of the samples of the changes I made:

Fixing Sorting:

Before:

```
const trips = await Trip.find().exec();
```

After:

```
// Sort trips dynamically
const sortBy = req.query.sortBy || 'start';
const direction = req.query.direction === 'desc' ? -1 : 1;

const trips = await Trip.find()
  .sort({ [sortBy]: direction }) // Dynamic sorting
  .exec();
```

Improvement:

- Organizes data logically (date, price, name, etc.)
- Gives user control over data order
- Demonstrates algorithmic thinking

Added Search:

Before

After

```
const trips = await Trip.find().exec();
```

```
const search = req.query.search || '';
const regex = new RegExp(search, 'i');

const query = {
  $or: [
    { code: regex },
    { name: regex },
    { resort: regex }
  ]
};

const trips = await Trip.find(query).exec();
```

What is better:

- Lets the users find specific data quickly
- Uses pattern matching
- Improves usability

Data Retrieval

Before

```
const tripsList = async (req, res) => {
  const trips = await Trip.find().exec();
  res.status(200).json(trips);
};
```

After

```
const page = Math.max(parseInt(req.query.page) || 1, 1);
const pageSize = Math.max(parseInt(req.query.pageSize) || 6, 1);
const skip = (page - 1) * pageSize;

const trips = await Trip.find()
  .skip(skip)       // skip previous results
  .limit(pageSize) // limit results per page
  .exec();
```

What is better:

- Prevents loading too much data at once

List of files updated:

- travlr/app_api/controllers/trips.js
- app_admin/src/app/services/trip-data.service.ts (new)
- app_admin/src/app/trip-listing/trip-listing.component.ts
- app_admin/src/app/trip-listing/trip-listing.component.html

- app_admin/src/app/models/trip-page.ts

Finnaly I stated to review my database design, I noticed that some data types and validation were not ideal for example storing perPerson as a string instead of a number. I updated it to a numeric type and added validation to ensure only valid, non-negative values are stored, along with improvements like trimming fields and adding timestamps. I also updated the seed data and frontend models to match the new structure, keeping everything consistent across the application. Below are some of the samples of the changes I made:

Changing String to Number:

Before:

```
perPerson: { type: String, required: true }
```

After:

```
perPerson: { type: Number, required: true, min: 0 }
```

Improvement:

- Uses the correct data type (number instead of string)
- Allows proper calculations and sorting
- Prevents invalid values like text or negative numbers

Added Timestamps to the Schema:

Before

```
const tripSchema = new mongoose.Schema({
  code: { type: String, required: true },
  name: { type: String, required: true },
  length: { type: String, required: true },
  start: { type: Date, required: true },
  resort: { type: String, required: true },
  perPerson: { type: Number, required: true },
  image: { type: String, required: true },
  description: { type: String, required: true }
});
```

After

```
const tripSchema = new mongoose.Schema({
  code: { type: String, required: true },
  name: { type: String, required: true },
  length: { type: String, required: true },
  start: { type: Date, required: true },
  resort: { type: String, required: true },
  perPerson: { type: Number, required: true, min: 0 },
  image: { type: String, required: true },
  description: { type: String, required: true }
},

  timestamps: true
}
);
```

What is better:

- Tracks when data is created and updated automatically
- Makes data easier to manage and audit

Validated data before saving

Before

```
const newTrip = new Trip({
  code: req.body.code,
  name: req.body.name,
  length: req.body.length,
  start: req.body.start,
  resort: req.body.resort,
  perPerson: req.body.perPerson,
  image: req.body.image,
  description: req.body.description
});

const savedTrip = await newTrip.save();
```

After

```
if (isNaN(Number(req.body.perPerson)) || Number(req.body.perPerson) < 0) {
  return res.status(400).json({
    message: 'perPerson must be a valid non-negative number'
  });
}

const newTrip = new Trip({
  code: req.body.code,
  name: req.body.name,
  length: req.body.length,
  start: req.body.start,
  resort: req.body.resort,
  perPerson: Number(req.body.perPerson),
  image: req.body.image,
  description: req.body.description
});

const savedTrip = await newTrip.save();
```

What is better:

- Prevents bad or invalid data from being stored
- Improves data integrity
- Provides clearer error messages to users

Updated seed data for new Schema

Before

After

What is better:

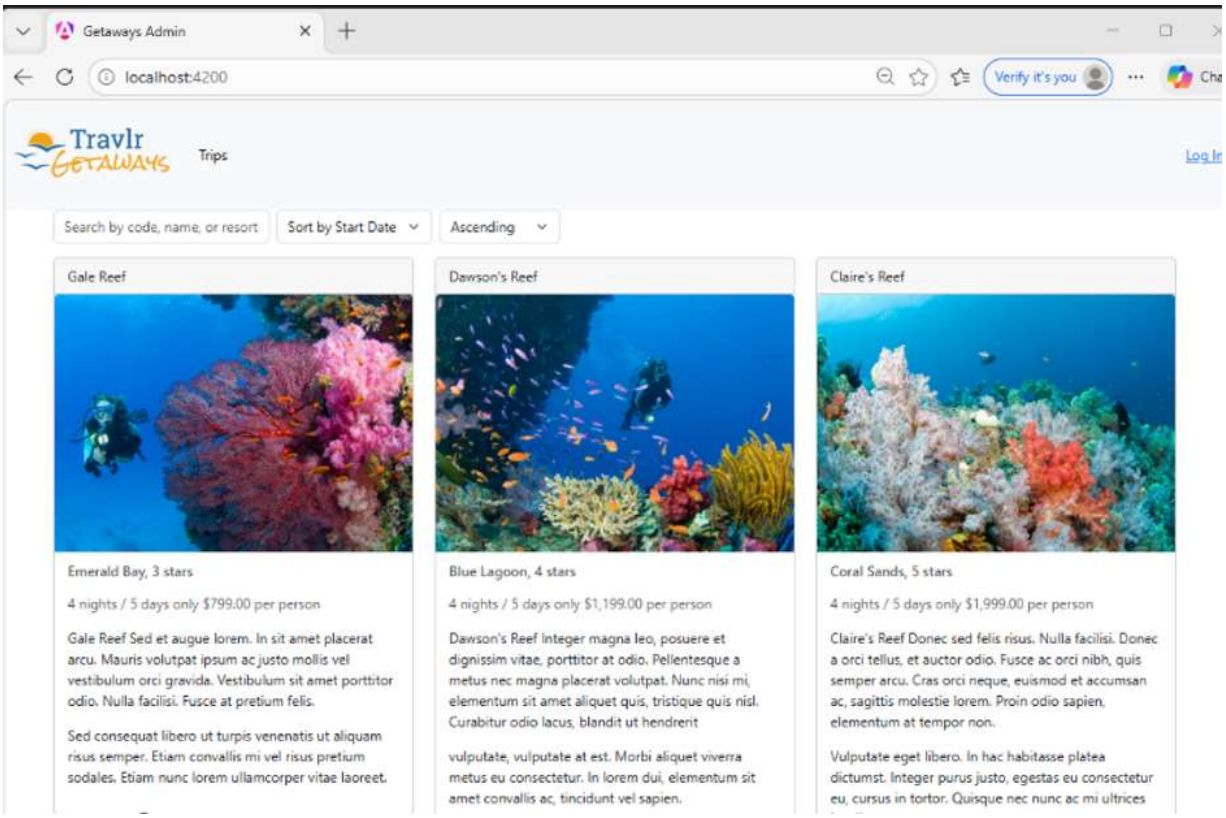
- Keeps data consistent with the database schema
- Prevents errors when inserting data
- Ensures smooth communication between backend and database

List of files updated:

- travlr/app_api/models/travlr.js
- travlr/app_api/controllers/trips.js
- travlr/app_api/routes/index.js
- travlr/app_api/models/seed.js
- travlr/data/trips.json
- app_admin/src/app/models/trip.ts
- app_admin/src/app/add-trip/add-trip.component.ts
- app_admin/src/app/edit-trip/edit-trip.component.ts

Reflection:

Enhancing these artifact really helped me understand what it means to improve a system, not just build one from scratch. I learned how important it is to use the right data types, add validation, and keep everything consistent across the frontend, backend, and database. One of the biggest challenges was making sure my changes when updating things like data types didn't break other parts of the app, so I spent a lot of time testing carefully and update multiple files. I also used feedback to focus more on structure, security, and making the code easier to maintain. In the end, the artifact is more reliable and realistic, and it helped me fully meet the outcomes related to improving software systems, while still continuing to grow in areas like optimization and data handling.



For a more in-depth detail of each change, please Download and review final capstone project code.